IU2641231920 – CPP80 – SUBMISSION REPORT

Project Name:	Cinema Ticket Booking System
Enrollment No.:	IU2641231920
Student Name:	Rhytham Patel
Project Code:	CPP80
Date of Submission:	13 October 2026

1. Project Summary

Problem Statement:

Managing student details, movie screenings, auditorium seat allocations, customer profiles, and ticket sales manually can be time-consuming and may result in errors, duplicated records, and difficulty retrieving information. The Cinema Ticket Booking System is a menu-driven C++ console application intended to organize cinema records and support adding, displaying, searching, updating, and deleting records. It also covers interactive seat selection, customer categorization with tiered discounts, reporting, and saving/loading data where supported by the implementation.

Objectives:

- Develop a structured, menu-driven C++ application using Object-Oriented Programming.
- Maintain movie, auditorium screen, customer, and ticket transaction records.
- Support adding, displaying, searching, updating, and deleting records.
- Organize auditorium seat assignments using interactive visual 2D grid layouts.
- Provide customer category discounts (Regular, VIP, Student) and organized reports.
- Use file handling to save records and load them for future use, where implemented.
- Reduce manual record-keeping and improve the organization of cinema operations.

Key Features:

- Movie record management, including search, update, duration, and price modification.
- Screen auditorium management, including screen name and seating capacity.
- Customer category management with polymorphic discounts (VIP and Student concessions).
- Interactive booking transaction with visual seat selection and conflict prevention.
- Search options across IDs, titles, and phone numbers via a menu-driven console interface.
- Report generation and file-based record persistence, where implemented.
- Modular C++ code organized into separate source and header files.

2. Class Summary

Class	Purpose / Responsibility	Key Data	Key Operations
Movie	Stores movie details and manages movie records.	Movie ID, title, genre, duration, rating, base price	Add/display/search/update/delete records
Screen	Maintains cinema screen and auditorium information.	Screen ID, name, rows, cols, dynamic seat matrix	Manage seat layout, book/cancel seat, display map
Customer	Abstract base class for all customer profiles.	Customer ID, name, contact number	Calculate discount (pure virtual), display customer
RegularCustomer	Represents standard tier cinema visitors.	Inherited customer credentials	Standard pricing evaluation (0% discount)
VIPCustomer	Represents premium loyalty members.	Loyalty points, customer credentials	20% loyalty discount calculation, add points
StudentCustomer	Represents student concession cardholders.	Student ID card number, contact info	10% concession discount calculation, verification
Ticket	Represents an individual booked cinema ticket.	Ticket ID, movie/screen ID, seat code, final price, date	Generate receipt card, display summary, serialize
FileManager	Handles persistent record storage and retrieval.	File/record data for movies, screens, customers, tickets	Save/load records where implemented
Cinema	Coordinates the main menu and application modules.	Module objects/records, dynamic ticket array	Control menu and coordinate operations

3. Project Flow

The system follows a menu-driven flow in which the user selects an operation, the relevant module processes the requested records, and supported changes may be saved to files.

Project Flow: USER → Main Menu → Movie / Screen / Customer Management → Seat Reservation / Reports → File Manager → Data Files

4. UML Deliverables

The report identifies a UML diagrams section. The following deliverables should be included or verified in the final project folder.

Diagram / Deliverable	Purpose	Status
Use Case Diagram	Shows the user and main cinema-management functions.	Include/verify
Class Diagram	Shows the main classes and their relationships.	Include/verify
Sequence Diagram 1	Shows the sequence of one supported operation.	Include/verify
Sequence Diagram 2	Shows the sequence of another supported operation.	Include/verify
Screenshots	Shows relevant source files and application execution.	Include/verify

5. C++ Concepts Demonstrated

Concept	Where Used / Evidence	Explanation
Classes and Objects	Movie, Screen, Customer, Ticket, Cinema and other modules.	Represent real-world cinema entities and structured records.
Encapsulation	Data and related operations grouped within classes.	Keeps related state and functions organized together.
Abstraction	Menu-driven console interface and module operations.	Allows users to interact without needing internal details.
File Handling	File Manager and record persistence, where implemented.	Uses C++ file streams to read and write saved records.
Modular Design	Main.cpp, Cinema.h/.cpp, Movie.h/.cpp, Screen.h/.cpp, Customer.h/.cpp.	Improves readability, organization and maintenance.
Inheritance	RegularCustomer, VIPCustomer, StudentCustomer derive from Customer.	Allows a class to reuse or extend another class's members.
Polymorphism	Pure virtual calculateDiscount() and getCustomerType() overridden.	Allows a common interface to behave differently through overriding.

File Handling

C++ file handling can retain cinema records after the program closes. ofstream is used for writing, ifstream for reading, and fstream for both reading and writing. The exact files and persistence behavior should match the submitted source code.

- Open a file in the required read/write mode.
- Write records when saving supported data.
- Read saved records when loading data.
- Close files after completing operations and handle errors where implemented.

6. Testing

Use the checklist below to verify the main functions. Record the actual result after running the submitted program.

Test ID	Operation	Input	Expected Output	Result
T01	Add movie	Valid movie details	Record stored and confirmation shown	Verify
T02	Display movies	Choose display option	Saved records displayed	Verify
T03	Search movie	Existing movie ID/Title	Matching record displayed	Verify
T04	Update movie	Edit an existing record	Updated details reflected	Verify
T05	Delete movie	Select existing record	Selected record removed	Verify
T06	Manage screen	Screen ID and seat grid	Screen record stored/displayed	Verify
T07	Customer registration	Customer details & category	Customer record stored/displayed	Verify
T08	Seat layout map	Select screen ID	Visual [O]/[X] grid displayed	Verify
T09	Ticket booking	Valid movie/seat/customer	Ticket issued with discount applied	Verify
T10	Invalid input	Unsupported option/data	Input handled appropriately	Verify
T11	Save and reload	Save, exit and restart	Saved data loads if implemented	Verify
T12	Generate report	Choose report option	Available information displayed	Verify

Note: The provided project report does not record actual pass/fail results for every test. Replace “Verify” with the observed result before submission.

7. Deployment

Language: C++

Development Environment: Visual Studio Code, as described in the project report.

Compiler: C++ compiler configured in the development environment; record the exact compiler/version used.

Source files identified: Main.cpp, Source/Cinema.h, Source/Cinema.cpp, Source/Movie.h, Source/Movie.cpp, Source/Screen.h, Source/Screen.cpp, Source/Customer.h, Source/Customer.cpp, Source/Ticket.h, Source/Ticket.cpp, Source/FileManager.h, Source/FileManager.cpp.

Execution: Compile the source files using the configured C++ compiler and run the resulting console application.

8. Final Submission Checklist

- Source folder with main.cpp and modular .h/.cpp files
- Data folder containing sample/generated data files
- Use Case Diagram and Class Diagram
- Sequence Diagram 1 and Sequence Diagram 2
- Working executable and screenshots
- README.md and ProjectReport.pdf

Final ZIP filename: IU2641231920_CPP80_Project.zip

9. Final Verification

Run the executable from the final submission folder. Verify add, display, search, update, delete, main transaction, report and save/load operations. Close and reopen the program and confirm that saved data is still available.

10. Student Declaration

I understand the submitted Cinema Ticket Booking System project and can explain its purpose, classes, data records, menu-driven operations, file handling, and the documentation included with the project.

Student Signature: _____